

FMCG Distribution Management System

Complete Technical Architecture, Python Source Code Breakdown, Database Design, and Operational Documentation for Authorized FMCG Dealership (Everest Spices & Colgate-Palmolive).

System Version: 2.0 (Production)	Primary Language: Python 3.14
Framework: Flask 3.1 & SQLAlchemy ORM	Database: SQLite (anupama_agencies.db)
Frontend: Jinja2, TailwindCSS, Chart.js	Documentation Date: September 2026

1. Executive Summary & Business Context

Anupama Agencies is an authorized FMCG super-stockist and distributor representing leading Indian multinational brands: **Everest Spices** (blended & pure spice formulations) and **Colgate-Palmolive India Ltd.** (oral healthcare and hygiene goods). Operating an FMCG dealership requires coordinated logistics between godown warehousing, beat delivery to hundreds of kirana retail outlets, staff attendance, advance cash disbursements, fleet fuel expenses, and damaged goods reconciliation.

The Challenge: Traditionally, distribution agencies rely on physical muster rolls, paper khata notebooks, and disconnected spreadsheets. This introduces critical operational errors:

- **Khata Slip Leakage:** Cash advances given to godown workers and drivers are frequently forgotten or dispute-prone during monthly wage disbursement.
- **Fuel & Fleet Opacity:** High diesel expenditures for mini-trucks and delivery vans lack odometer verification and receipt cross-checking.
- **Beat Reconciliation Delays:** Morning crate dispatches (Everest crates and Colgate cartons) are not tied to driver/helper pairs, obscuring delivery bottlenecks.
- **Manufacturer Claim Loss:** Transit damage and expiry returns from retailers are delayed in submission to depot, causing credit note claims to expire.

The Solution: A high-performance, Python-driven centralized web portal that brings complete digital transparency, real-time calculation, automated muster roll generation, printable payment vouchers, and Excel-compatible data exports.

2. System Architecture & Tech Stack

The application follows a clean **Model-View-Controller (MVC)** architectural pattern with an ~80% Python backend footprint and a ~20% lightweight frontend presentation tier. No heavyweight Node.js build pipelines are required, ensuring instant portability across Windows, Linux, and cloud environments.

Layer	Components & Technology	Key Responsibilities
-------	-------------------------	----------------------

Presentation (View)	Jinja2 Templates, HTML5, Vanilla CSS (custom.css), TailwindCSS (CDN), Lucide Icons, Chart.js	Render server-side dashboards, dynamic forms, mobile-responsive navigation, Chart.js fuel trends, and printable wage vouchers.
Controller (Flask Routes)	Flask 3.1 (app.py) with 24 route handlers, Jinja context filters, file upload handler	Request dispatching, payload validation, business calculations (wages, deductions, fuel diffs), session flash notifications.
Data Access (ORM)	SQLAlchemy 2.0 & Flask-SQLAlchemy 3.1.1 (models.py)	Object Relational Mapping, cascade deletions, lazy loading, computed model properties, and aggregation queries.
Persistence (Database)	SQLite 3 (anupama_agencies.db), database.py	Transactional zero-config file database, automated schema generation, and realistic FMCG dealership mock data seeding.
Utility & Export	utils.py, Python standard csv, io, datetime	Indian Rupee (INR) formatting, secure file upload hashing, and UTF-8 BOM CSV streaming for Microsoft Excel.

3. Detailed Python Code Breakdown

The Python backend comprises five primary files: `models.py`, `database.py`, `utils.py`, `app.py`, and `smoke_test.py`. Below is the systematic breakdown of every component.

3.1 Data Models & Relationships (`models.py`)

`models.py` defines 7 relational database entities utilizing Flask-SQLAlchemy. It incorporates cascading relationships, foreign key constraints, and dynamic Python calculation methods.

Model Name	Key Attributes & Types	Relationships & Methods
Worker	<code>id</code> , <code>name</code> , <code>phone</code> , <code>emergency_contact</code> , <code>role</code> , <code>join_date</code> , <code>daily_wage</code> , <code>monthly_salary</code> , <code>status</code> , <code>photo_path</code>	Relationships: <code>attendances</code> , <code>advances</code> , <code>salary_settlements</code> (cascade='all, delete-orphan'). Methods: <code>total_advances_pending</code> , <code>days_worked_in_month()</code> , <code>attendance_summary_in_month()</code> .
Attendance	<code>id</code> , <code>worker_id</code> (FK), <code>date</code> , <code>status</code> , <code>notes</code>	Enforces unique constraint: (<code>worker_id</code> , <code>date</code>). Statuses: Present (1.0), Half Day (0.5), Paid Leave (1.0), Absent (0.0).
Advance	<code>id</code> , <code>worker_id</code> (FK), <code>request_date</code> , <code>payment_date</code> , <code>amount_requested</code> , <code>amount_paid</code> , <code>payment_mode</code> , <code>reason</code> , <code>status</code> , <code>settlement_id</code> (FK)	Tracks khata loans. Statuses: Pending, Approved, Settled. Links directly to <code>SalarySettlement</code> upon wage deduction.
Vehicle	<code>id</code> , <code>vehicle_number</code> (unique), <code>vehicle_name</code> , <code>vehicle_type</code> , <code>fuel_type</code> , <code>assigned_driver_id</code> (FK), <code>status</code>	Represents fleet assets (Tata Ace, Mahindra Bolero, Ape Auto). Relationships: <code>fuel_expenses</code> , <code>dispatches</code> .
FuelExpense	<code>id</code> , <code>vehicle_id</code> (FK), <code>driver_id</code> (FK), <code>date</code> , <code>fuel_type</code> , <code>amount_spent</code> , <code>liters</code> , <code>odometer_reading</code> , <code>receipt_image</code> , <code>notes</code>	Stores fuel fill logs with physical receipt image path and odometer tracking.
DispatchBeat	<code>id</code> , <code>dispatch_date</code> , <code>beat_name</code> , <code>vehicle_id</code> (FK), <code>driver_id</code> (FK), <code>helper_id</code> (FK), <code>crates_everest</code> , <code>cartons_colgate</code> , <code>status</code> , <code>notes</code>	Logs morning retail deliveries per commercial beat. Statuses: Dispatched, Completed.
DamagedReturn	<code>id</code> , <code>return_date</code> , <code>brand</code> , <code>product_name</code> , <code>retailer_name</code> , <code>batch_no</code> , <code>expiry_date</code> , <code>quantity</code> , <code>unit</code> , <code>reason</code> , <code>claim_status</code> , <code>credit_note_amount</code> , <code>credit_note_number</code>	Tracks retailer returns for Everest & Colgate. Pipeline: Pending Inspection → Submitted to Depot → Credit Note Received.
SalarySettlement	<code>id</code> , <code>worker_id</code> (FK), <code>month_year</code> , <code>days_present</code> , <code>half_days</code> , <code>paid_leaves</code> , <code>absent_days</code> , <code>effective_days</code> , <code>daily_wage</code> , <code>gross_earnings</code> , <code>advances_deducted</code> , <code>bonus_incentive</code> , <code>net_payout</code> , <code>settlement_date</code> , <code>payment_mode</code> , <code>payment_reference</code>	Stores immutable monthly wage settlement snapshots and generates formal printable wage payment vouchers.

3.2 Database Initialization & Realistic Seeding (database.py)

database.py provides `init_db(app)` which ensures storage directories exist (static/uploads/workers and static/uploads/receipts), executes `db.create_all()` to instantiate the SQLite schema, and automatically seeds realistic data if the tables are empty:

- **5 Godown Staff:** Senior Drivers (Ramesh Kumar, Amit Sharma), Loaders (Suresh Yadav), Helpers (Manoj Verma), and Supervisors (Rajesh Patel) with varying daily wages (Rs. 500 to Rs. 800).
- **3 Fleet Vehicles:** Tata Ace Gold (Diesel), Mahindra Bolero Maxi Truck (Diesel), and Piaggio Ape Auto (CNG).
- **Multi-day Attendance Matrix:** Pre-populated for the active month with Present, Half Day, Absent, and Paid Leave records.
- **Live Khata Advances:** Real-world advance records (medical, school fees, repair) in Approved and Settled states.
- **Beat Dispatches & Damage Claims:** Pre-configured route trips and credit note claims for both Everest Spices and Colgate-Palmolive.

3.3 Core Utilities & Export Engine (utils.py)

utils.py encapsulates mission-critical helper functions utilized across routes and Jinja templates:

1. Indian Rupee (INR) Formatter (`format_inr`): Standard Python locale formatting frequently misbehaves across Windows and Linux server environments. `format_inr` implements the exact Indian numbering system (separating the last 3 digits, followed by groupings of 2 digits for thousands, lakhs, and crores):

Input: 154250.5 → Output: Rs. 1,54,250.50. Supports negative amounts, null protection, and 2-decimal precision.

2. Secure File Uploader (`save_uploaded_file`): Validates file extensions (PNG, JPG, JPEG, WEBP), generates collision-free timestamped filenames, saves to designated uploads directories, and returns relative web URLs.

3. Microsoft Excel-Compatible CSV Streamer (`export_csv_response`): Builds on Python's in-memory `io.StringIO` and standard `csv.writer`. Critically, it prepends the **UTF-8 Byte Order Mark (BOM)** so that Microsoft Excel correctly recognizes Indian Rupee characters and Unicode text without character corruption.

3.4 Application Routing & Business Controller (app.py)

app.py serves as the application nerve center, spanning 961 lines of meticulously structured Python code. It configures Flask, registers custom Jinja filters, injects global date variables, and exposes 24 endpoints categorized into 9 operational functional modules:

Module	Route Endpoint	Method	Description & Logic
1. Dashboard	/	GET	Aggregates real-time KPIs: worker headcount, today's attendance tallies, pending advance sum, month-over-month fuel expenditure with % difference, active dispatches, pending damaged claims, and generates dynamic 6-month fuel trend arrays for Chart.js.
2. Workers	/workers /workers/add /workers/<id> /workers/<id>/edit	GET POST GET POST	Directory of staff with role & status filtering, search filter, photo upload, monthly attendance summary, advance history, and WhatsApp/Call integration.
3. Attendance	/attendance /attendance/save /attendance/monthly	GET POST GET	Daily bulk attendance register with 'Mark All Present' shortcut. Monthly muster roll matrix view (days 1–31) calculating effective days and attendance %.
4. Advances	/advances /advances/add /advances/<id>/status	GET POST POST	Staff khata ledger. Records cash/UPI/bank advances. Computes Total Given, Total Pending, and Total Settled. Live status toggle.
5. Fuel & Fleet	/fuel /fuel/add /fuel/vehicles /fuel/vehicles/add	GET POST GET POST	Tracks fleet logs (Tata Ace, Bolero, Ape Auto). Logs liters, amount, odometer reading, and fuel receipt image upload with image preview modal.
6. Dispatches	/dispatches /dispatches/add /dispatches/<id>/status	GET POST POST	FMCG daily route delivery tracker. Records beat name, vehicle, driver, helper, Everest Spices crates, and Colgate cartons dispatched.
7. Returns	/returns /returns/add /returns/<id>/update-claim	GET POST POST	Damaged & expiry log for Everest and Colgate. Tracks batch numbers, retailer names, depot claim statuses, and credit note numbers.
8. Settlements	/settlements /settlements/process /settlements/voucher/<id>	GET POST GET	Automated wage calculator applying monthly attendance, daily wages, advance deduction cascade, bonus incentive, and generates printable payment voucher.
9. CSV Exports	/export/attendance /export/advances /export/fuel /export/settlements /export/returns	GET	Five instant one-click downloadable Excel-compatible CSV reports generated on-the-fly via streaming response with UTF-8 BOM encoding.

3.5 Quality Assurance & Automated Verification (smoke_test.py)

smoke_test.py incorporates Python's standard unittest framework to execute 6 rigorous test suites verifying the integrity of the application:

1. `test_database_seeded`: Confirms that initial seed data contains at least 5 workers, 3 vehicles, and active advances.
2. `test_all_pages_render_200`: Simulates HTTP GET requests across all 11 core routes, asserting HTTP 200 OK status codes.
3. `test_attendance_save`: Verifies POST data submission to `/attendance/save` and asserts persistence in SQLite.
4. `test_advance_and_balance`: Validates that recording a new advance increments the worker's pending balance accurately.
5. `test_settlement_computation_and_voucher`: Simulates salary processing, verifies advance deduction logic, and confirms voucher rendering.
6. `test_csv_exports`: Downloads all 5 CSV export endpoints, asserting text/csv MIME headers and non-zero byte payloads.

4. Mathematical Formulas & Business Logic

The application replaces manual accounting guesswork with deterministic mathematical calculations codified in Python:

4.1 Effective Working Days Calculation

Implemented in `Worker.days_worked_in_month(year, month)`. A half-day is credited as 0.5, while full days and paid leaves receive 1.0 credit:

```
Effective Working Days = Present Count + Paid Leave Count + (Half Day Count × 0.5)
```

4.2 Monthly Salary Settlement & Khata Deduction Engine

Implemented in `/settlements/process` in `app.py`. The gross earnings are computed directly from the worker's agreed daily wage rate, and advances are automatically deducted up to the authorized deduction limit:

```
Gross Earnings = Effective Working Days × Daily Wage Rate
Net Payout = MAX( 0.0 , (Gross Earnings - Advances Deducted) + Bonus Incentive )
```

Advance Cascade Logic: When a monthly salary is settled with advance deductions, the system iterates through the worker's oldest approved advances. If an advance is smaller than or equal to the remaining deduction amount, it is flagged as Settled. If an advance is larger than the remaining deduction, it is marked as Settled and an automated remainder advance record is created for the remaining balance. This guarantees that khata ledgers never lose track of fractional advance balances.

4.3 Fleet Fuel Expenditure Variance

Computed on the executive dashboard to provide instant operational cost control over delivery vans and trucks:

```
Fuel Variance (%) = [ (Current Month Expense - Previous Month Expense) / Previous Month Expense ] × 100
```

5. Frontend Architecture & Design Aesthetics

The user interface is engineered according to enterprise UI standards, featuring custom branding for Anupama Agencies:

- **Bespoke Brand Colorway:** Dual brand themes combining **Everest Warm Red** (#C41E3A) and **Colgate Trust Blue** (#004B87) with slate neutrals (#F8FAFC).
- **Template Inheritance (base.html):** Establishes a responsive dual-navigation pattern: a sticky collapsible left sidebar on desktop and an animated mobile drawer for tablet/smartphone warehouse use.
- **Micro-Interactions (custom.css & app.js):** Smooth hover lifts on metric cards (card-hover), status badges for muster rolls (present green, half-day amber, absent red, leave blue), auto-dismissing flash notifications, and image modals for instant receipt inspection.
- **Interactive Wage Calculator:** Live JavaScript calculation in `templates/settlements/index.html` dynamically updates net payouts in real time as managers adjust advance deduction and bonus inputs.
- **Printable Payment Voucher (voucher.html):** Utilizes dedicated CSS @media print rules. Strips navigation, sidebars, and backgrounds, outputting a legal payment voucher complete with dual signature blocks for Godown Manager and Worker.

6. Installation, Configuration & Run Guide

6.1 Prerequisites

- Python 3.10, 3.11, 3.12, 3.13, or 3.14 installed on Windows, macOS, or Linux.
- Standard pip package manager.

6.2 Step-by-Step Installation

1. Open PowerShell or Terminal in the project root directory:
`cd "c:\python project for anupama agenices"`
2. Install required Python packages:
`pip install -r requirements.txt`
3. Launch the application server:
`python app.py`
4. Access the web dashboard in any browser:
`http://127.0.0.1:5000`

6.3 Running Quality Assurance Tests

Execute automated smoke tests anytime to guarantee zero regressions:
`python smoke_test.py`

6.4 Production Deployment Recommendations

For production deployment on an on-premise warehouse server or cloud VPS (Ubuntu/Debian):

- **WSGI HTTP Server:** Run behind **Gunicorn** or **Waitress**: `waitress-serve --port=5000 app:app` or `gunicorn -w 4 -b 0.0.0.0:5000 app:app`.
- **Reverse Proxy:** Terminate SSL/TLS via **Nginx** or **Caddy** with HTTPS certificates.
- **Database Scaling:** SQLite supports thousands of daily transactions effortlessly. If agency operations expand to multiple multi-city depots, the SQLAlchemy URI can be switched to PostgreSQL with zero model code modifications.
- **Automated Backups:** Schedule a daily backup cron job for `instance/anupama_agencies.db` and the `static/uploads/` directory.

Document Prepared For: Anupama Agencies (Distributor Operations)	Status: Approved for Production Deployment
Technical Lead: AI Senior Python Solutions Architect	Distribution Partners: Everest Spices & Colgate-Palmolive India